지원 직무: DevOps / AI Consultant

010-8626-1074



이　　름 : 백승우

생 년 월 일 : 2001.03.16

주　　소 : 서울특별시 노원구 공릉동 509-10

연 락 처 : (H/P) 010-8626-1074 / (E-mail) swbaik01@gmail.com

학　　력 : 서울과학기술대학교 ITM 전공 4학년

영국 Northumbria University - Information Technology Management (연계학위)

경력 및 강점요약

○ AI 모델 개발과 서비스 구현부터 데이터 처리·클라우드 배포까지 수행 가능한 DevOps·AI 융합 역량 보유

- 영국 Northumbria University 연계 학위 과정 수행으로 영어회화 가능
- 5인 AI 플랫폼 프로젝트의 팀장으로 서비스 기획, 시스템 아키텍처, API·DB 구조 및 AI 파이프라인 설계 총괄
- LangChain·LangGraph를 활용한 단계형 AI 워크플로와 ChromaDB 기반 RAG·유사 사례 검색 구조 설계 및 구현
- 사용자 입력 검증, NLP 구조화, 이미지 품질검사, 표준 단가 조회, LLM 결과 생성·검증을 연결한 AI 서비스 파이프라인 구축
- 약 2,000건의 데이터 구성과 5개 AI 모델의 성능·처리속도·예외 대응 평가를 통해 서비스 적용 모델 선정 및 품질 기준 수립
- React·FastAPI·AI Service 분리 아키텍처와 GitHub Actions·Docker·AWS 기반 CI/CD 및 독립 배포 환경 구축
- HDFS·PySpark 기반으로 약 30GB 데이터를 처리하고 VMware 저장공간을 20GB에서 200GB로 확장해 대용량 데이터 처리 환경 개선
- TensorFlow Lite 객체 인식 모델을 Android 앱에 임베딩하고 Room DB·Firestore·GPT API를 연결한 모바일 AI 서비스 구현
- PyTorch 기반 의료 이미지 분류 모델을 설계해 테스트 정확도를 약 94%에서 99%로 향상
- Spring Boot·JPA·MySQL 기반 REST API 개발과 Swagger 문서화, Docker Compose 기반 개발 환경 구축
- 팀장 및 다수의 팀 프로젝트 경험을 바탕으로 요구사항 구조화, 기술 검증, 파트 간 연동을 포함한 Jira 기반 프로젝트 일정·우선순위·산출물 관리 역량 보유

Skill Set

- **AI / Data : Pytorch, Langchain, Langgraph, Hadoop(HUE), Py-Spark, Tenfsorflow**
- **Frontend : React, Vite, JavaScript, HTML, Tailwind CSS, Figma Design, Node.js, Express.js, PWA**
- **Backend : Java, Python, Kotlin, Spring, FastAPI, Android Studio, Gradle, FireBase**
- **DataBase : MYSQL, SQLite, FireStore**
- **Cloud : AWS, Hugging Face, Docker, Linux**
- **Management Tools : Git, Jira, Monday, Notion**

학력사항

'21. 02 ~ '28.02	**서울과학기술대학교 / ITM 전공**
	Northumbria University / Information Technology Management

자격증

자격명	취득일	기관명
Cos Pro Python 2급	2021.07.18	YBM
IELTS - Overall Band Score 6.0	2024.11.19	British Council, IDP, Cambridge University

교육사항

교육명	기간	교육기관명
Google Developer Groups	`24.09~`25.03	GDG Campus Korea
[IBMxRedHat] AI Transformation – AX Academy	`26.02~`26.08	대한상공회의소

포트폴리오

프로젝트 기간	2025.09 ~ 2025.12	참여인원	4인
프로젝트 개요	Big Data Processing 전공 과목 팀 프로젝트로, 뉴욕 택시 운행 데이터와 교통 속도 데이터를 활용해 택시 이동 시간의 불확실성을 분석한 빅데이터 처리 프로젝트		
링크	발표자료: https://docs.google.com/presentation/d/1ggftALXueGg85IgzxbEmJn8PwVdj5SMI4ARLWOIZ8QM/edit?usp=sharing		
담당 역할	HDFS 데이터 적재, PySpark 기반 전처리, 분석용 Mart 생성, TLI/CI 산출 로직 구현	개발환경 / Tech stack	**Hadoop, HDFS, PySpark, Spark, SQL, Python, VMware Linux VM**
내용	약 30GB 규모의 뉴욕 택시·교통 데이터를 Hadoop HDFS에 적재하고 PySpark로 정제·분석한 프로젝트. 데이터 파이프라인을 Bronze–Silver–Gold Layer로 구성해 원천 데이터 보존, 전처리, 파생 변수 생성, 최종 집계를 단계적으로 수행. Pickup/Dropoff 시간, 이동 거리, LocationID, 교통 속도를 기반으로 15분 단위 시간 블록과 Borough 정보를 생성하고, 이동 시간 변동성을 나타내는 TLI(Taxi Luck Index)와 혼잡도 지표인 CI 산출.		

구현기능 및 개발과정

대용량 데이터 처리를 위해 Hadoop HDFS 기반 저장 환경 구축. VMware 저장 공간 부족과 Permission Denied 문제가 발생해 디스크를 20GB에서 200GB로 확장하고, Linux 파티션 마운트와 HDFS Data Directory 재설정을 통해 해결.

Windows의 CSV 데이터를 VM으로 전송한 뒤 hdfs dfs -put 명령어로 HDFS /data/ 경로에 적재. PySpark에서 CSV를 Parquet 형식으로 변환해 저장 효율과 처리 성능 개선.

전처리 과정에서 음수 이동 시간을 이상치로 제거하고 불필요한 요금 컬럼을 삭제. Pickup과 Dropoff 시간 차이로 duration_seconds를 계산하고, 15분 단위 Time Block과 AM/PM 컬럼 생성. Taxi Zone Lookup 테이블과 조인해 PULocationID와 DOLocationID를 Borough 정보로 변환.

Spark SQL 집계 함수를 활용해 Borough, Hour, 15분 Block 기준 TLI와 CI 계산. 결과 Parquet 파일을 Pandas로 병합해 TLI_FINAL.csv, CI_FINAL.csv 형태의 분석용 Mart 생성.

성과

약 30GB 데이터를 안정적으로 적재·처리할 수 있는 HDFS·PySpark 기반 파이프라인 구축. Parquet 변환과 컬럼 프루닝을 적용해 저장 효율과 처리 성능을 개선하고, Bronze–Silver–Gold 구조를 통해 데이터 처리 단계의 일관성과 재사용성 확보.

저장 공간 부족 문제를 직접 해결하고, 원천 데이터부터 최종 분석 Mart까지 이어지는 데이터 엔지니어링 전 과정을 구현.

느낀 점

빅데이터 분석에서는 모델링 이전에 안정적인 저장 환경과 처리 파이프라인을 설계하는 것이 중요함을 체감. 저장 공간, 파일 포맷, 컬럼 구성, 데이터 품질이 처리 성능과 분석 신뢰도에 직접적인 영향을 준다는 점을 학습.

Borough 단위 분석은 지역 범위가 넓어 세부 교통 패턴을 설명하는 데 한계가 있었음. 향후 Zone 또는 Road-level 단위로 분석 범위를 세분화하고, Kafka와 Airflow를 도입해 수동 배치 파이프라인을 자동화할 계획.

포트폴리오

프로젝트 기간	**26.05 ~ 26.07**	**참여인원**	**5인**
프로젝트 개요	IBM x Red Hat 부트캠프 파이널 팀 프로젝트로, 사진과 자연어 설명을 기반으로 집수리 AI 견적을 생성하고 리스크 리포트와 시공자 매칭까지 연결하는 플랫폼		
링크	**발표자료:** https://canva.link/m5n77930osszn9p **GitHub (Organization) :** https://github.com/TUKTAKxAI		
담당 역할	**팀장, 프로젝트 전체 기능 설계, 서비스 플로우 설계, DB/API 구조 설계, CI/CD 배포 구조 설계, AI 견적 파이프라인 설계 총괄, 프로젝트 관리**	**개발환경 / Tech stack**	**React, Vite, Tailwind CSS, FastAPI, Python, MySQL, SQLAlchemy, Alembic, LangGraph, LangChain, ChromaDB, Hugging Face Transformers, OpenCV, Pandas, Docker, GitHub Actions, AWS ECS, ECR, S3, CloudFront, Route 53, ALB, RDS MySQL, Jira**
내용	집수리 시장의 가격 불투명성과 사용자 정보 부족 문제를 해결하기 위해 개발한 AI 기반 집수리 플랫폼. 사용자가 수리 대상 사진과 설명을 입력하면 AI가 대상, 문제 유형, 예상 작업, 가격 범위, 소요 시간을 분석하고 리스크 리포트와 시공자 매칭까지 연결. AI 견적 → 리스크 분석 → N:N 시공자 매칭 → 역경매 견적 비교 → 시공 선택으로 이어지는 집수리 의사결정 전 과정을 하나의 서비스 플로우로 구성.		

구현기능 및 개발과정

팀장으로서 MVP 범위를 회원가입·로그인, AI 견적, 리스크 리포트, 매칭 요청, 시공자 견적 발송·선택, 마이페이지로 정의하고 고객과 시공자의 사용자 흐름 및 개발 우선순위 정리.

백엔드는 FastAPI 기반으로 구축하고 인증, 사용자, 시공자, AI 견적, 리스크 리포트, 매칭, 견적, 시공, 알림 기능을 도메인별 라우터로 분리. SQLAlchemy와 Alembic으로 MySQL 스키마와 마이그레이션을 관리하고, AI 견적부터 실제 시공까지 연결되는 도메인 모델 설계.

AI 견적 서비스는 LangGraph 기반으로 입력 검증, 이미지 품질 검사, 텍스트 유효성 검사, NLP 구조화, 단가 조회, 유사 사례 검색, 견적 생성, 결과 검증 단계를 구성. 각 처리 단계와 실패 지점을 추적할 수 있도록 파이프라인화.

Rule-Engine으로 욕설, 스팸, 무의미 문장, 수리 무관 입력을 1차 차단하고 애매한 요청은 REVIEW_REQUIRED로 분류. 테스트 결과 정확도 93.21%, Macro F1 0.8862, 비정상 입력 정상 통과율 0.00% 기록.

KLUE-RoBERTa-base를 활용해 사용자 설명에서 main_category, object_label, problem_label, repair_task, missing_info를 추출하도록 설계. 이미지 입력은 OpenCV로 품질을 검사하고, 이미지 임베딩과 ChromaDB를 활용해 기존 수리 사례와 비교.

견적 생성 시 전체 단가표 대신 매칭된 표준 단가와 유사 사례 최대 3개만 LLM에 전달해 토큰 사용량과 임의 가격 생성을 줄임. 가격 최소·최대 관계, 단가 허용 범위, 필수 JSON Key를 검증하는 후처리 규칙 적용.

프론트엔드는 GitHub Actions, Vite, S3, CloudFront 기반으로 배포하고, 백엔드는 Docker 이미지를 ECR에 Push한 뒤 ECS에 배포하도록 CI/CD 구성. AWS 인증은 OIDC 방식으로 설계해 장기 Access Key 사용을 방지.

Jira에서 기능을 Epic, Sprint, Task로 구분해 관리하고, API·DB·프론트엔드·AI·배포 작업의 진행 상황과 의존성을 조율.

성과

AI 견적, 리스크 분석, 시공자 매칭, 역경매 견적 비교까지 이어지는 집수리 플랫폼의 전체 구조 설계. Frontend, Backend, AI Service를 분리해 기능 책임과 서비스 간 결합도를 명확히 관리.
FastAPI 백엔드와 LangGraph 기반 AI 견적 파이프라인의 설계 및 구현 방향 총괄. Rule-Engine 성능 평가에서 정확도 93.21%, Macro F1 0.8862, 비정상 입력 정상 통과율 0.00% 달성.
GitHub Actions, Docker, ECR, ECS, S3, CloudFront 기반 배포 구조를 구성하고, Jira를 활용해 각 파트가 병렬로 개발할 수 있는 협업 환경 구축.

느낀 점

팀장으로 프로젝트를 진행하며 기능 수보다 서비스 흐름, 데이터 구조, API 계약, AI 처리 기준을 먼저 정의하는 것이 중요함을 체감.
AI 기능 역시 단순 모델 호출이 아니라 입력 검증, 구조화, 검색, 생성, 후처리 검증이 분리된 안정적인 파이프라인으로 설계해야 결과 신뢰도를 관리할 수 있음을 학습.
또한 팀장은 일정만 관리하는 역할이 아니라 모든 파트가 동일한 명세와 기준을 바탕으로 개발할 수 있도록 방향을 조율해야 함을 경험. 기능 명세서, API 명세서, ERD, AI 흐름도, CI/CD 구조를 선행 정리해 병렬 개발과 서비스 일관성을 확보.

포트폴리오

프로젝트 기간	**25.09 ~ 25.12**	**참여인원**	**3인**
프로젝트 개요	Mobile Programming 전공 과목에서 한 학기 동안 진행한 팀 프로젝트로, TensorFlow Lite 객체 인식 모델을 활용해 어린이가 주변 사물을 촬영하며 영어 단어를 학습할 수 있도록 만든 Android 모바일 앱		
링크	**발표자료 : https://canva.link/wxyy36liv3mgimr** **GitHub (Organization) : https://github.com/chalkak-itm**		
담당 역할	**TensorFlow Lite 객체 인식 모델 조사 및 선정, Android 앱 내 TFLite 모델 임베딩, ObjectDetectionHelper 구현, 객체 label·confidence score·bounding box 추출 로직 구현, 객체 인식 결과와 영어 단어 학습 플로우 연동**	**개발환경 / Tech stack**	**Kotlin, Android, XML View, TensorFlow Lite, TFLite Task Vision, Firebase Authentication, Firestore, Firebase Functions, Firebase Cloud Messaging, Room DB, Google Credential Manager, OpenAI API, STT, TTS, Gradle**
내용	어린이가 일상 속 사물을 직접 촬영하며 영어 단어를 학습할 수 있도록 개발한 모바일 앱. 반복 암기 방식에서 벗어나 사진 촬영, 객체 인식, 단어·예문 학습, 발음 연습, 퀴즈 복습으로 이어지는 학습 흐름을 설계. 촬영된 이미지는 앱에 내장된 TensorFlow Lite 모델로 분석하며, 인식 결과를 Label, Confidence Score, Bounding Box 형태로 제공. 사용자가 선택한 단어의 뜻과 예문은 Room DB에서 조회하고, 신규 단어는 Firebase Functions와 GPT API를 통해 생성해 Firestore와 로컬 DB에 저장. TTS·STT 발음 학습, 사진 기반 학습 기록, 간격 반복 퀴즈, FCM 알림, 학습 캘린더 기능을 추가해 촬영부터 반복 학습까지 앱 내에서 완료할 수 있도록 구성.		

구현기능 및 개발과정

TensorFlow Lite 객체 인식 모델을 assets/1.tflite로 앱에 포함해 서버 연결 없이 기기 내에서 동작하도록 구현. TFLite Task Vision의 ObjectDetector를 활용한 ObjectDetectionHelper를 개발해 모델 로드, 이미지 변환, 추론 과정을 모듈화.

인식 결과는 Label, Confidence Score, Bounding Box를 포함한 DetectionResult로 변환. 최대 감지 객체 수를 2개, Confidence Threshold를 0.3으로 설정하고 이미지 위에 Bounding Box를 표시해 결과를 직관적으로 확인할 수 있도록 구성.

Room DB에는 촬영 사진, 감지 단어, 예문, 마지막 학습 시간을 저장하고, photo_logs, detected_objects, example_sentences 테이블을 관계형으로 연결. Firestore의 단어 데이터는 앱 실행 시 로컬 DB와 동기화해 기존 학습 데이터를 빠르게 조회하도록 설계.

신규 단어나 예문이 부족한 경우에만 Firebase Functions를 통해 GPT API를 호출해 한국어 뜻과 영어 예문 3개를 생성. 결과를 Firestore와 Room DB에 함께 저장해 동일 단어에 대한 중복 호출 방지.

퀴즈는 마지막 학습 시간이 오래된 단어를 우선 출제하고, 정답 시 lastStudied를 갱신하며 오답은 같은 세션에서 다시 출제하도록 구현. TTS 발음 듣기, STT 발음 비교, FCM 학습 알림, 연속 학습일 및 Quick Snap 센서 기능도 함께 적용.

성과

TensorFlow Lite 기반 온디바이스 객체 인식 기능을 구현하고, 촬영 결과를 단어 조회, GPT 기반

예문 생성, 발음 학습, 퀴즈 복습과 연결해 사진 기반 영어 학습 플로우 완성. Room DB와 Firestore를 결합한 Local-first 구조로 조회 속도와 데이터 재사용성을 높였으며, 필요한 경우에만 GPT API를 호출해 중복 요청 감소. 퀴즈 데이터는 한 번에 조회한 뒤 메모리에서 그룹화해 N+1 쿼리 문제 개선.
느낀 점
모바일 AI 서비스에서는 모델 정확도뿐 아니라 추론 결과를 실제 사용자 경험과 자연스럽게 연결하는 설계가 중요함을 학습. 객체 인식 로직을 Helper로 분리하며 AI 기능을 재사용 가능한 모듈로 구성하는 경험 확보. 또한 로컬 저장소, 클라우드 동기화, 음성·센서 기능이 함께 동작하는 환경에서 데이터 흐름과 호출 시점을 명확히 설계해야 함을 체감. Room DB와 Firestore를 함께 활용하며 Local-first 구조와 API 호출 최적화의 필요성을 이해.

포트폴리오

프로젝트 기간	**25.10 ~ 25.11**	**참여인원**	**1인 (개인)**
프로젝트 개요	Deep Learning 전공 과목에서 진행한 CNN 미니 프로젝트로, 망막 OCT 이미지를 입력받아 CNV, DME, DRUSEN, NORMAL 4개 클래스로 분류하는 CNN 기반 망막 질환 이미지 분류 모델을 구현했다.		
링크	**NoteBook 파일 링크 :** https://drive.google.com/file/d/1QKTKYXi_Ljb7BH3T6wlZ2P0IbtnUNoPB/view?usp=sharing		
담당 역할	**OCT 이미지 데이터셋 구성, CNN 모델 설계, PyTorch 학습 파이프라인 구현, 데이터 전처리 및 증강, Baseline/Improved 모델 성능 비교, Confusion Matrix 분석**	**개발환경 / Tech stack**	**Python, PyTorch, Torchvision, CNN, Jupyter Notebook, scikit-learn, NumPy, Matplotlib, Seaborn, CUDA/GPU**
내용	망막 OCT 이미지는 질환별 차이가 미세해 단순 분류보다 특징 추출 성능이 중요하다고 판단. CNV, DME, DRUSEN, NORMAL 4개 클래스를 대상으로 Baseline CNN과 SE-Residual 기반 개선 모델을 설계하고 동일 조건에서 성능 비교. OCT2017 데이터셋의 학습 이미지 83,484장을 8:2 비율로 Train/Validation으로 분리하고, 별도 Test 이미지 1,000장으로 최종 검증. 이미지는 Grayscale 1채널, 224×224 크기로 변환하고 정규화 적용. 학습 데이터에는 RandomResizedCrop, HorizontalFlip, Rotation을 적용해 위치와 촬영 조건 변화에 대한 일반화 성능 강화.		

구현기능 및 개발과정

Baseline 모델은 Conv2D–BatchNorm–ReLU–MaxPool 구조의 Conv Block 4개와 AdaptiveAvgPool2D, Linear 계층으로 구성.
개선 모델은 Residual Connection과 Squeeze-and-Excitation을 결합한 SE-ResBlock 기반 CNN으로 설계. Residual 구조로 Gradient 흐름을 안정화하고, SE Block으로 질환 판별에 중요한 채널 특징에 집중하도록 구성.
PyTorch 기반 학습 파이프라인을 직접 구현해 DataLoader로 데이터를 분리하고 Epoch별 Loss와 Accuracy 기록. 학습 완료 후 Test Set 기준 Accuracy, Precision, Recall, F1-score, Confusion Matrix를 산출해 모델별 성능 비교.
Baseline은 일부 질환 간 혼동이 발생했으나, 개선 모델은 전체 클래스에서 안정적인 분류 성능을 보였으며 특히 시각적으로 유사한 CNV와 DME 간 오분류 감소.

성과

Baseline CNN은 약 94%, SE-Residual 개선 모델은 약 99%의 Test Accuracy 기록. 약 5%p의 성능 향상을 달성했으며, Test 이미지 1,000장 중 오분류를 14건 수준으로 감소.
NORMAL 클래스는 250장을 모두 정확히 분류. 모델 비교 실험을 통해 의료 이미지 분류에서는 네트워크 깊이뿐 아니라 Gradient 안정성과 채널별 특징 선택이 성능에 큰 영향을 준다는 점 확인.

느낀 점

CNN 성능은 계층 수를 늘리는 것보다 데이터 특성에 적합한 구조를 선택하는 것이 중요함을 체감. 질환별 차이가 작은 OCT 이미지에서는 BatchNorm, Residual Connection, SE Attention이 학습 안정성과 특징 추출 성능 향상에 효과적임을 확인.
Accuracy뿐 아니라 클래스별 Precision, Recall, Confusion Matrix를 함께 분석해야 실제 오분류 패턴을 파악할 수 있음을 학습. 데이터 전처리부터 모델 설계, 성능 비교와 해석까지 딥러닝 실험의 전체 흐름 경험.

포트폴리오

프로젝트 기간	**24.09 ~ 24.12**	**참여인원**	**5인**
프로젝트 개요	**Open Source Software 전공 과목에서 진행한 팀 프로젝트로, 학생들이 강의 자료와 과제 접근 방식을 공유하고 질문·답변을 남길 수 있는 캠퍼스 기반 지식 공유 플랫폼**		
링크	**발표자료 : https://canva.link/u1aqszltue6oiv4** **Github : https://github.com/mmuhunn/CampusShare**		
담당 역할	**Spring Boot 백엔드 API 구현, JPA 엔티티 설계, 자료 공유 API 구현, Q&A API 구현, MySQL 연동, Swagger API 문서화**	**개발환경 / Tech stack**	**Java, Spring Boot, Spring Web, Spring Data JPA, Spring Security, MySQL, Gradle, Swagger/OpenAPI, React, Axios, Docker Compose**
내용	학생들이 수업 자료, 과제 팁, 질문과 답변을 공유할 수 있도록 개발한 학업 커뮤니티 플랫폼. 강의 노트와 시험 자료를 등록·검색·다운로드하고, 질문과 답변을 주고받을 수 있는 구조로 설계. 자료 공유 기능에서는 학번, 제목, 내용, 파일 정보를 입력해 게시물을 등록하고 전체 조회와 제목 검색 기능 제공. 다운로드 시 카운트를 증가시켜 자료 활용도를 확인할 수 있도록 구성.		

구현기능 및 개발과정

Spring Boot 기반 REST API를 구현하고 User, Share, Question, Answer 도메인으로 분리. 각 도메인은 Controller–Service–Repository 계층으로 구성하고 Spring Data JPA를 활용해 MySQL과 연동.
회원가입과 로그인은 학번과 비밀번호를 기반으로 구현하고, 자료·질문·답변 작성 시 사용자 정보를 연결. 자료 게시물은 등록, 조회, 검색, 다운로드 카운트 증가 기능을 제공.
Q&A는 Question과 Answer를 별도 도메인으로 구성해 질문 등록과 답변 등록·조회·삭제가 가능하도록 구현. 질문과 답변의 관계를 통해 게시판형 질의응답 구조 구성.
Swagger/OpenAPI를 적용해 API 경로, 요청 파라미터, 응답 구조를 문서화하고 React 프론트엔드와 연동 가능한 형태로 설계. MySQL은 Docker Compose로 실행하고 JPA 설정과 SQL 로그를 구성해 로컬 개발 및 테스트 환경 마련.
오픈소스 과목 프로젝트로 README, License, Contributing Guide, Code of Conduct를 함께 관리하며 협업과 외부 기여를 고려한 저장소 구조 구성.

성과

Spring Boot와 JPA를 활용해 회원, 자료 공유, 검색, 다운로드 집계, 질문·답변 기능을 포함한 핵심 백엔드 API 구현.
도메인과 계층별 책임을 분리해 유지보수 가능한 구조를 구성하고, Swagger와 Docker Compose를 적용해 프론트엔드 연동 및 로컬 테스트 편의성 향상.
6명의 팀원이 참여하고 209개의 커밋을 기록한 프로젝트를 통해 Git 기반 협업과 오픈소스 프로젝트 운영 방식 경험.

느낀 점

Spring Boot 프로젝트에서는 CRUD 구현뿐 아니라 도메인과 계층별 책임을 명확히 나누는 것이 중요함을 학습. 프론트엔드와 연동하며 API 경로, 파라미터, 응답 형식을 일관되게 관리해야 협업 효율이 높아진다는 점을 체감.
또한 사용자, 자료, 질문, 답변 간 관계를 설계하며 JPA 엔티티 연관관계와 Service 계층의 필요성을 이해. 기능 구현 외에도 문서화와 기여 가이드를 관리하는 과정이 협업 가능한 프로젝트를 만드는 데 중요하다는 점을 배움.

자기소개서

Q1. 성장과정에 대해 서술하시오.

저는 대학교 2학년 때 개발 역량을 높이기 위해 대학생 개발자들이 기술을 학습하고 프로젝트 경험을 공유하는 커뮤니티인 Google Developer Groups에 가입했습니다. 당시 Java와 Spring에 대한 지식이 부족했을 뿐만 아니라, MVC(Model-View-Controller) 패턴에서 각 계층이 어떤 역할을 담당하고 요청과 응답이 어떻게 이동하는지도 명확히 이해하지 못한 상태였습니다.

이러한 상황에서 Spring을 활용한 백엔드 개발 세션에 참여했고, 단순히 예제 코드를 따라가는 것이 아니라 Spring 애플리케이션의 구조와 실행 흐름을 이해하는 것을 목표로 삼았습니다. 처음에는 Controller, Service, Repository의 역할을 구분하지 못해 기능을 수정할 때마다 오류가 발생했고, 의존성 주입과 데이터베이스 연동 과정에서도 어려움을 겪었습니다. 이를 해결하기 위해 세션 이후 예제 프로젝트를 다시 구현하며 각 계층의 책임을 따로 정리했습니다. 또한 요청이 Controller로 들어와 Service의 비즈니스 로직과 Repository의 데이터 처리를 거쳐 응답으로 반환되는 과정을 직접 추적했습니다. 오류가 발생했을 때는 로그를 확인해 원인을 기록하고, 관련 공식 문서와 팀원들의 코드를 비교하며 해결했습니다.

그 결과 MVC 구조를 기반으로 Spring 백엔드 기능을 구현했으며, 마지막에는 Dockerfile을 작성해 애플리케이션을 이미지로 만들고 컨테이너 환경에서 실행하는 배포 과정까지 완료했습니다. 이를 통해 개발 결과물이 실제 서비스가 되기 위해서는 코드 구현뿐 아니라 실행 환경의 일관성과 배포 과정까지 고려해야 한다는 점을 배웠습니다. 이후 저는 새로운 기술을 마주했을 때 개념만 학습하는 데 그치지 않고, 구현·실행·배포까지 연결해야 온전히 이해할 수 있다는 가치관을 갖게 되었습니다. 이 태도를 바탕으로 DevOps 직무에서도 개발과 운영의 흐름을 정확히 파악하고, 안정적이고 반복 가능한 배포 환경을 구축하겠습니다.

Q2. 갈등 해결 경험에 대해 서술하시오.

저는 협업 과정에서 의견이 충돌할 때 상대방을 설득하기보다, 우려의 원인을 분석하고 검증 가능한 근거를 제시해 합의점을 만듭니다. 6명이 참여한 지역 기반 행사 탐색 프로젝트에서 팀장을 맡았을 당시, 메인 화면에 대한민국 지도를 배치하고 10개 권역을 클릭할 때마다 선택 애니메이션과 해당 지역의 행사 정보를 보여주는 기능을 기획했습니다. 그러나 팀원은 일반 이미지로는 지역별 클릭 영역을 구분하기 어렵고, 제한된 일정 안에 애니메이션까지 구현하는 것은 불가능하다며 다른 화면 구성을 제안했습니다.

저는 팀장의 의견을 일방적으로 밀어붙이지 않고, 팀원이 우려한 핵심이 기술 자체보다 구현 방법과 소요 시간을 알 수 없다는 점이라고 판단했습니다. 먼저 일반 이미지 대신 지도를 SVG 형식으로 변환하면 각 지역의 벡터 경로를 개별 요소로 분리할 수 있다는 방법을 조사했습니다. 이후 SVG의 path 값을 추출해 React 컴포넌트에 적용하고, 지역별 클릭 이벤트와 선택 상태에 따른 확대·색상 변경 애니메이션을 연결했습니다. 전체 기능을 바로 개발하기보다 한 개 지역을 대상으로 간단한 시제품을 제작해 회의에서 직접 시연했고, 동일한 구조를 나머지 9개 권역에 반복 적용할 수 있다는 구현 방식과 작업 범위를 함께 설명했습니다.

작동하는 결과를 확인한 팀원들은 기존 기획을 유지하는 데 동의했고, 역할을 나누어 일정 내에 10

개 권역의 선택 기능과 애니메이션을 완성했습니다. 이를 통해 핵심 사용자 경험을 포기하지 않으면서도 팀 내 의견 충돌을 해결할 수 있었습니다. 이 경험으로 갈등은 말로 이기는 것이 아니라, 상대의 우려를 구체화하고 최소 기능을 빠르게 검증할 때 해결된다는 점을 배웠습니다. DevOps 업무에서도 기술 도입이나 배포 방식에 이견이 발생한다면 작은 단위의 검증 환경을 구축하고, 실행 결과와 데이터를 근거로 안정적인 의사결정을 이끌겠습니다.

Q3. 핵심역량에 대해 서술하시오.(AI_Consultant)

AI 컨설팅 업무의 핵심은 단순히 성능이 좋은 모델을 제안하는 것이 아니라, 고객의 문제를 분석해 실행 가능한 AI 서비스 구조로 전환하고 성능·비용·운영 가능성까지 검증하는 역량이라고 생각합니다. 저는 수리 AI 견적 및 시공자 매칭 플랫폼 개발 프로젝트의 팀장으로서 서비스 기획부터 AI 플로우, 성능평가, 배포 구조까지 전 과정을 주도했습니다.

프로젝트 초기에는 'AI로 수리 견적을 제공한다'는 아이디어만 존재했으며, 어떤 데이터를 입력받고 어떤 모델을 거쳐 결과를 생성할지 정해지지 않은 상태였습니다. 저는 이를 ① AI 예상 견적, ② RAG 기반 리스크 리포트, ③ 실시간 N 역경매 매칭의 3개 핵심 서비스로 구체화했습니다. 특히 AI 견적 서비스를 이미지·설명 입력, OpenCV 품질검사, CLIP 이미지 임베딩, ChromaDB 유사 사례 검색, 텍스트 구조화, 표준 단가표 조회, Vision LLM 분석, 최종 견적 계산 단계로 세분화했습니다. 또한 반복적인 LLM 호출 비용을 줄이기 위해 유사도가 높은 사례는 Vision LLM을 생략하는 분기 구조도 설계했습니다.

모델 선정 과정도 감이 아닌 데이터로 판단했습니다. 사용자 문장을 7개 필드로 구조화한 약 2,000건의 데이터를 설계하고, 정상 요청 70%, 모호한 요청 20%, 비수리 요청 10%로 구성했습니다. 이후 KLUE-RoBERTa, KoELECTRA, KoT5, Qwen2.5, EXAONE 등 5개 모델을 대상으로 4단계 성능평가를 설계했습니다. 출력 성공률과 형식 준수율부터 필드별 정확도, Macro-F1, Exact Match, 예외 입력 처리, 추론 시간까지 비교한 결과 KLUE-RoBERTa를 최종 모델로 선정했으며, 평균 정확도 86.4%와 평균 처리시간 0.014초를 확인했습니다. 동시에 예외 입력 성능을 보완하기 위한 Rule-based Pre-filter 도입 방안도 제시했습니다.

마지막으로 프론트엔드는 GitHub Actions, S3, CloudFront로, 백엔드와 AI 서비스는 Docker, ECR, ECS, ALB, RDS 기반으로 배포 흐름을 설계해 AI 결과가 실제 서비스로 제공되는 구조까지 완성했습니다. 이 경험을 통해 AI 컨설팅은 불명확한 요구사항을 구조화하고, 선택 기준을 수치화하며, 기술을 운영 가능한 시스템으로 연결하는 일임을 배웠습니다. 이러한 역량을 바탕으로 고객의 업무 프로세스를 분석하고, PoC 설계부터 모델 평가와 구축 로드맵까지 구체적인 AI 도입 방안을 제시하겠습니다.

Q3. 핵심역량에 대해 서술하시오.(DevOps)

DevOps 직무의 핵심 역량은 애플리케이션의 특성에 맞는 클라우드 환경을 설계하고, 빌드·테스트·배포 과정을 자동화해 개발 결과물을 안정적으로 운영하는 능력이라고 생각합니다. 저는 수리 AI 견적 및 시공자 매칭 플랫폼 개발 프로젝트의 팀장으로서 AWS 아키텍처와 CI/CD 파이프라인을 직접 설계하고 구축했습니다.

프로젝트 초기에는 React 기반 프론트엔드, FastAPI 백엔드, 별도의 AI 서비스가 각각 개발되고 있었지만, 배포 환경과 서비스 간 통신 방식이 정해지지 않아 팀원마다 실행 환경이 달랐습니다. 또한

수동 배포를 반복할 경우 이미지 버전 관리와 배포 누락이 발생할 가능성이 있었습니다. 저는 세 서비스를 독립적으로 배포하면서도 하나의 사용자 요청 흐름으로 연결하는 것을 목표로 설정했습니다. 먼저 프론트엔드는 정적 파일 제공에 적합하도록 S3에 배포하고, CloudFront를 CDN으로 연결해 콘텐츠 전송 구조를 설계했습니다. 백엔드와 AI 서비스는 Docker로 컨테이너화한 뒤 ECR에서 이미지를 관리하고, ECS에서 각각 독립된 서비스로 실행하도록 구성했습니다. 외부 요청은 Route 53의 도메인과 ALB를 통해 FastAPI 서비스로 전달하고, 데이터는 RDS MySQL에 저장하도록 총 7개의 주요 AWS 서비스를 연결했습니다. 컨테이너 상태는 /health 엔드포인트와 Target Group 헬스 체크를 통해 확인하도록 설정했습니다.

배포 과정도 GitHub Actions로 자동화했습니다. 프론트엔드는 Git Push 이후 테스트와 빌드를 수행하고 S3 업로드와 CloudFront 캐시 갱신까지 이어지도록 구성했습니다. 백엔드는 테스트, Docker 이미지 빌드, ECR Push, ECS 서비스 업데이트가 순차적으로 실행되도록 파이프라인을 구축했습니다. 민감한 AWS 인증 정보와 환경변수는 GitHub Secrets로 분리해 코드에 노출되지 않도록 관리했습니다.

그 결과 팀원이 코드를 반영하면 동일한 절차와 실행 환경으로 배포되는 구조를 완성했고, 프론트엔드·백엔드·AI 서비스를 독립적으로 수정하고 배포할 수 있게 했습니다. 이 경험을 통해 DevOps는 단순히 서버를 생성하는 업무가 아니라 개발 속도, 운영 안정성, 보안과 확장성을 함께 설계하는 역할임을 배웠습니다. 입사 후에도 서비스 특성에 맞는 AWS 아키텍처와 자동화 파이프라인을 구축해 반복 작업과 배포 위험을 줄이는 데 기여하겠습니다.

Q4. 직무 수행 경험에 대해 서술하시오.

저는 공공데이터 기반 지역 행사 탐색 및 AI 코스 추천 웹 개발 프로젝트에서 팀장을 맡아,, 개별 기능을 하나의 서비스로 통합했습니다. 초기에는 프론트엔드와 백엔드가 기능별로 개발되고 있었지만, 공공데이터 수집과 조회, 사용자 인증, AI 추천과 결과 저장까지의 연결 기준이 명확하지 않았습니다. 외부 API와 OpenAI API의 응답 실패가 전체 서비스 중단으로 이어질 가능성도 있었습니다.

저는 API 명세서와 ERD를 기준으로 인증·사용자, 공공데이터 동기화, 행사 조회, AI 추천, 사용자 상호작용 영역을 분리했습니다. 이 중 회원가입·로그인과 AI 추천 기능을 담당해 지역·날짜·키워드 조건으로 후보 행사를 조회하고, OpenAI API가 3개의 코스를 추천하도록 구현했습니다. 추천 결과의 행사 ID가 실제 후보 목록에 존재하는지 검증하고, 유효한 결과가 3개 미만이면 규칙 기반 추천으로 전환하는 fallback 로직을 적용했습니다. 저장 시에도 행사 ID와 순서의 중복을 검사해 잘못된 데이터가 반영되지 않도록 했습니다.

백엔드는 FastAPI의 Router·Service·CRUD 계층을 분리하고 비동기 데이터베이스 세션을 활용해 기능 책임을 구분했습니다. 또한 공공데이터 동기화 로직을 별도 서비스로 분리하고, 정기 동기화를 고려한 스케줄러 구조를 마련했습니다. 실행 환경은 React 프론트엔드, FastAPI 백엔드, MySQL을 각각 컨테이너화한 뒤 Docker Compose로 통합하고 AWS EC2에 배포했습니다. 환경변수 예시와 Docker 실행 문서를 함께 관리해 팀원별 로컬 환경 차이를 줄이고, 동일한 명령으로 전체 서비스를 실행할 수 있도록 했습니다.

그 결과 행사 검색·필터링부터 상세 조회, 좋아요·북마크·리뷰, AI 코스 추천과 저장까지 이어지는 전체 흐름을 실제 웹 서비스 형태로 완성했습니다. 이 경험을 통해 AI 기능의 직무 수행에서 중요한

것은 모델이나 API를 호출하는 것만이 아니라, 입력 데이터 검증·실패 시 대체 경로·저장 무결성·실행 환경을 함께 설계하는 것임을 배웠습니다. 입사 후에도 DevOps 직무에서 AI 구현은 외부 모델의 불확실성을 제어하는 검증·fallback 구조를 적용하고, CI/CD 구축은 개발 환경을 컨테이너로 표준화해 안정적으로 배포·운영할 수 있는 서비스 기반을 구축하겠습니다.

Q5. 문제해결 경험에 대해 서술하시오.

저는 주어진 개발 환경의 한계를 그대로 받아들이기보다, 시스템 구조를 분석하고 필요한 환경으로 직접 개선한 경험이 있습니다. 빅데이터 분석 프로젝트를 수행할 당시 VMware 기반 Linux 환경과 과제용 가상머신 이미지가 제공됐지만, 기본 디스크 용량은 약 19.6GB에 불과했습니다. 반면 프로젝트에서 사용한 교통 데이터는 압축 상태에서도 약 9.8GB, 압축 해제 후에는 26.4GB였으며, 추가로 3.7GB 규모의 택시 데이터를 HDFS에 적재해야 했습니다. 총 30GB 이상의 데이터를 처리해야 했기 때문에 기본 설정만으로는 저장공간이 부족했고, 데이터 적재 과정이 중단됐습니다.

저는 문제의 원인이 분석 코드가 아니라 가상머신의 스토리지 구성에 있다고 판단해 VMware와 Linux 파일 시스템 구조를 점검했습니다. 먼저 VMware에 200GB 규모의 신규 가상 디스크를 추가하고, Linux에서 해당 디스크를 인식시킨 뒤 파티션과 마운트 경로를 구성했습니다. 이후 HDFS가 확장된 디스크를 저장공간으로 활용하도록 설정하고, NameNode가 사용하는 데이터 디렉터리도 신규 디스크로 이전했습니다. 또한 Windows 로컬의 데이터를 scp와 SSH 파이프 방식으로 가상머신에 전송한 뒤, hdfs dfs −put 명령어를 이용해 26.4GB 교통 데이터와 3.7GB 택시 데이터를 HDFS에 적재했습니다. 마지막으로 HUE에서 Hive 외부 테이블을 생성해 분석 가능한 환경까지 완성했습니다.

그 결과 제공된 기본 이미지의 환경적 제약을 해결하고, 데이터를 축소하거나 삭제하지 않은 상태로 수집·적재·전처리·분석·시각화 과정을 모두 수행해 최종 평가에서 **A+**를 받았습니다. 이 경험을 통해 애플리케이션의 문제처럼 보이는 현상도 실제 원인은 운영체제, 스토리지 또는 데이터 처리 환경에 있을 수 있다는 점을 배웠습니다. DevOps 업무에서도 장애를 단편적으로 처리하지 않고 인프라 계층부터 원인을 분석해, 서비스 규모에 적합한 실행 환경을 능동적으로 구축하겠습니다.

Q6. 지원 동기 및 입사 후 포부에 대해 서술하시오

기업이 AI와 클라우드 기술을 도입하는 목적은 새로운 기술을 사용하는 데 그치지 않고, 업무 효율과 서비스 안정성을 높여 실질적인 성과를 만드는 데 있다고 생각합니다. 이를 위해서는 특정 기술만 구현하는 인력보다 요구사항을 분석하고, 적합한 기술을 선택한 뒤 검증·배포·운영까지 연결할 수 있는 인력이 필요합니다. 저는 AI 모델, 애플리케이션, 클라우드 인프라를 하나의 서비스 생명주기로 연결해 본 경험을 바탕으로 이러한 역할을 수행할 수 있다고 판단해 지원했습니다.

저를 채용해야 하는 이유는 AI 과제를 모델 검증에서 끝내지 않고, 실제 서비스와 운영 환경까지 연결할 수 있기 때문입니다. 많은 AI 프로젝트가 높은 모델 성능을 확보하고도 불명확한 요구사항, 개발 조직 간 연동 문제, 배포 환경의 부재로 인해 실사용 단계까지 이어지지 못합니다. 저는 사진과 자연어 설명을 기반으로 집수리 견적을 생성하고 리스크 분석과 시공자 매칭까지 제공하는 AI 플랫폼 프로젝트에서 5명 팀의 팀장을 맡아 이러한 문제를 직접 해결했습니다. 추상적인 아이디어를 3개의 핵심 서비스와 단계별 데이터 흐름으로 구체화하고, 프론트엔드·백엔드·AI 담당자가 동일한 기준으로 개발할 수 있도록 API와 아키텍처를 설계했습니다. 모델 선정에서도 단순 성능 수치만 보지 않고 정확도, 예외 처리, 추론 시간과 서비스 적용 가능성을 기준으로 5개 모델을 4단계에 걸쳐 비교

했습니다. 이후 GitHub Actions와 AWS 기반 CI/CD 환경까지 구축해 검증된 AI 기능을 실제 사용 가능한 서비스로 전환했습니다. 따라서 저는 기획, AI 검증, 개발, 배포 사이의 단절을 줄이고, 기업의 AI 과제가 PoC에 머무르지 않도록 실행 단계까지 이끌 수 있습니다.

입사 초기에는 조직의 서비스 구조와 배포 절차, 고객의 업무 프로세스를 빠르게 파악하고 반복 작업과 장애 가능성이 높은 구간을 구체화하겠습니다. DevOps 업무에서는 컨테이너 기반 실행 환경과 CI/CD 파이프라인을 표준화하고, 로그·모니터링·장애 대응 체계를 개선해 배포 안정성과 운영 효율을 높이겠습니다. AI 컨설팅 업무에서는 모델을 먼저 제안하기보다 업무 목표, 데이터 품질, 평가지표, 비용과 운영 조건을 기준으로 PoC를 설계하겠습니다. 장기적으로는 기술 검토와 제안에 그치지 않고 구축·배포·운영 개선까지 책임지며, 기업의 AI와 클라우드 투자가 지속 가능한 비즈니스 성과로 이어지도록 기여하겠습니다.

위 내용은 사실과 다름이 없습니다.

2026년 07월 29일

작성자: 백승우